Networks and Causal Inference Short Course (Part 2)

2026-03-11

Networks and Causal Inference Short Course

Part 2: Introduction to Network Science

In this part, we will use simulated network data similar to the Transmission Reduction Intervention Project in Athens, Greece

1. Load in the nodes and load in the edges data file. Note: we use two separate files: one to save the nodes and corresponding attributes and one to save all the connections.

```
library(readxl)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
nodes <- read.csv("TRIPsim_nodes.csv")
edges <- read.csv("TRIPsim_edges.csv")
```

2. Create a simple *igraph* object with the nodes and edges data, and compute simple characterization. Note that the graph is undirected (directed = FALSE).

```
library(igraph)
```

```
##
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union
```

```
## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
```

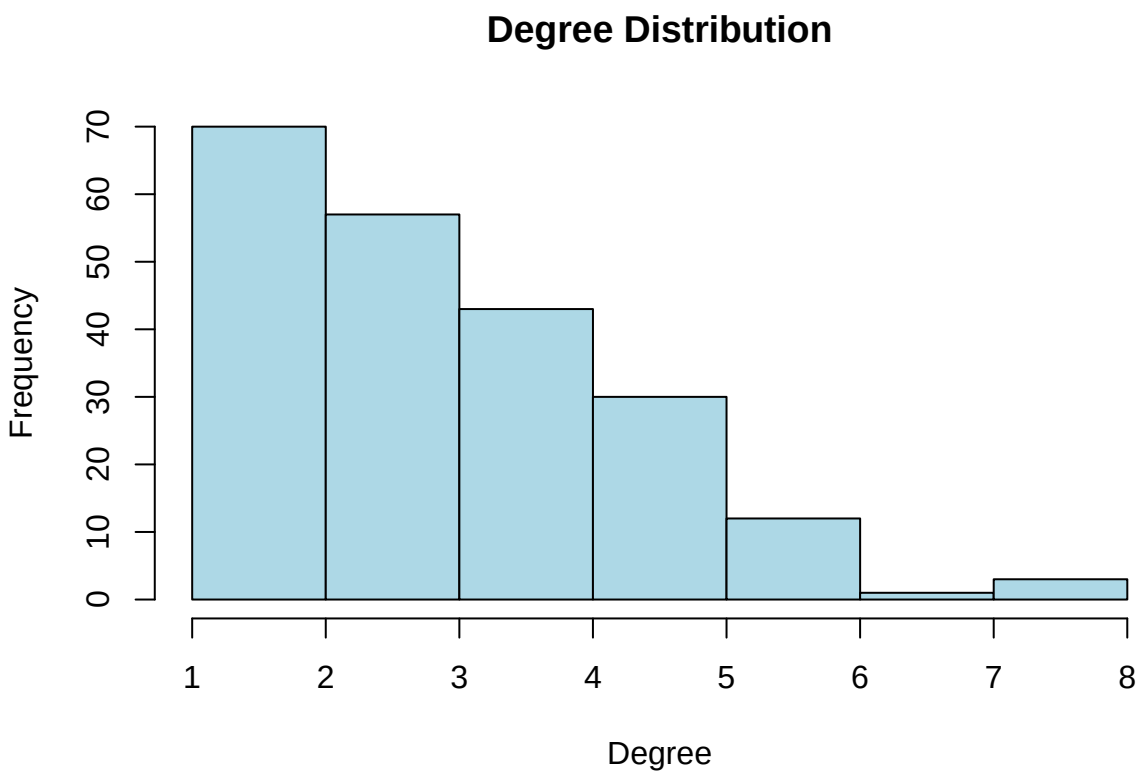
```
## The following object is masked from 'package:base':
##
##     union
```

```
my_network <- graph_from_data_frame(d = edges, vertices = nodes, directed = FALSE)
my_network
```

```
## IGRAPH eeaf12c UN-- 216 352 --
```

```
## + attr: name (v/c), num_nn (v/n), component (v/n), posneg (v/n), date
## | (v/n), share (v/n), education (v/n), employment (v/n), age (v/n),
## | avg_posneg (v/n), avg_date (v/n), avg_share (v/n), avg_education
## | (v/n), avg_employment (v/n), avg_age (v/n), raneff (v/n), treatment
## | (v/n), num_tnn (v/n), num_utnn (v/n), outcome (v/n)
## + edges from eeaf12c (vertex names):
##  [1] 1 --6  2 --6  1 --7  2 --7  3 --7  3 --8  6 --8  2 --9  10--11 1 --12
## [11] 3 --13 1 --14 2 --14 7 --14 1 --15 9 --15 1 --17 5 --17 12--17 1 --18
## [21] 5 --18 7 --18 16--18 2 --19 4 --19 2 --20 7 --20 11--20 17--20 14--21
## [31] 16--21 17--22 18--22 21--22 1 --23 3 --24 4 --24 10--24 11--24 9 --25
## + ... omitted several edges
```

```r
# number of nodes (order)
order_mnet<-vcount(my_network)

# number of edges (size)
size_mnet<-ecount(my_network)

# computing the density of the obtained network
density_mnet<-edge_density(my_network)

# compute graph degree for all nodes in the network
node.degree <- igraph::degree(my_network)
# average node degree and standard deviation
node.degree_avg <- mean(node.degree)
node.degree_sd <-sd(node.degree)

# illustrate graph degree distribution with histogram
hist(node.degree, col="light blue", xlab="Degree", ylab="Frequency", main="Degree Distribution")
```



```r
which.max(node.degree)
```

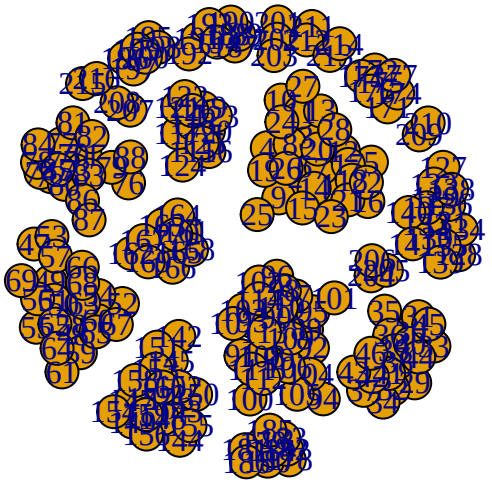
```
## 1
## 1
```

```r
# Number of connected components
num_components_mnet <- igraph::components(my_network)$no

# Compute transitivity of the network
transitivity_mnet <-igraph::transitivity(my_network)

# Degree assortativity
assortativity_mnet<- igraph::assortativity_degree(my_network)


# ------------------------------
# Print results
# ------------------------------
results_my_network <- list(
  Order = order_mnet,
  Size = size_mnet,
  Density = density_mnet,
  Average_Degree = node.degree_avg,
  SD_Degree =node.degree_sd,
  Connected_Components = num_components_mnet,
  Transitivity = transitivity_mnet,
  Degree_Assortativity = assortativity_mnet
)

results_my_network
```

```
## $Order
## [1] 216
##
## $Size
## [1] 352
##
## $Density
## [1] 0.01515935
##
## $Average_Degree
## [1] 3.259259
##
## $SD_Degree
## [1] 1.545408
##
## $Connected_Components
## [1] 20
##
## $Transitivity
## [1] 0.2395437
##
## $Degree_Assortativity
## [1] -0.0376786
```

3. Visualize the obtained network.

```
plot(my_network)
```



How about a better picture? First, select the layout and record the coordinates for future visualizations.

- 'vertex.label' parameter specifies node labels, 'NA' removes labels
- 'vertex.size' parameter specifies node sizes
- 'vertex.color' parameter specifies node colors
- 'vertex.shape' parameter specifies shapes of the modes

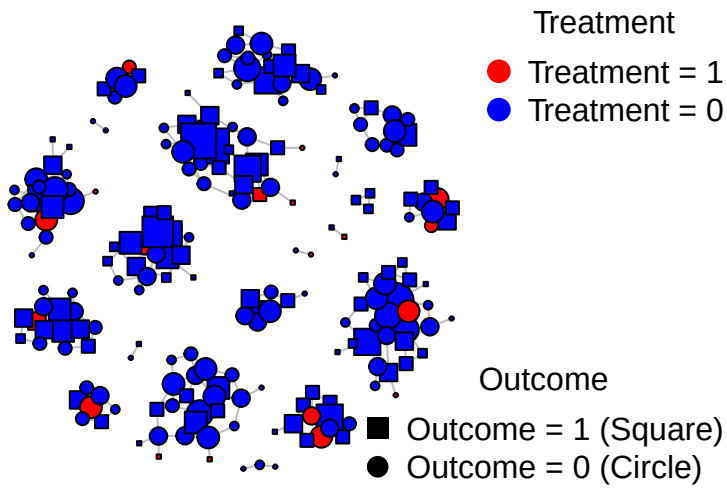
Changing labels, sizes, and colors of the vertices can help to uncover more information, for example, about the node attribute that you want to visualize. Note: the values of these parameters can be the same for all nodes or specified for each node individually.

In the following example, we assign node colors based on treatment status (alserted/not alterted).

```
# save the coordinates for the network
layout.coordinates <- igraph::layout_with_fr(my_network)

# ------------------------------------------------------------
# Assign node colors based on treatment status
# treatment = 1  -> red
# treatment = 0  -> blue
# ------------------------------------------------------------

V(my_network)$color <- ifelse(
  nodes$treatment == 1,
  "red",
  "blue"
)

# ------------------------------------------------------------
# Assign node shapes based on outcome status
# outcome = 1 -> square
# outcome = 0 -> circle
# ------------------------------------------------------------

V(my_network)$shape <- ifelse(
  nodes$outcome == 1,
  "square",
```

```
  "circle"
)

# ------------------------------------------------------------
# Plot the network
# ------------------------------------------------------------

plot(
  my_network,
  vertex.label = NA,
  vertex.color = V(my_network)$color,
  vertex.shape = V(my_network)$shape,
  vertex.size = 2*node.degree, # proportional to degree
  edge.width = 1,
  edge.color = "gray",
  layout = layout.coordinates,
  main = "Network by Treatment and Outcome",
  sub = "Node size proportional to degree"
)

# ------------------------------------------------------------
# Add legends
# ------------------------------------------------------------

# Legend for treatment (colors)
legend(
  "topright",
  legend = c("Treatment = 1",
             "Treatment = 0"),
  col = c("red", "blue"),
  pch = 19,
  pt.cex = 1.5,
  bty = "n",
  title = "Treatment"
)

# Legend for outcome (shapes)
legend(
  "bottomright",
  legend = c("Outcome = 1 (Square)",
             "Outcome = 0 (Circle)"),
  pch = c(15, 16),   # 15 = square, 16 = circle
  pt.cex = 1.5,
  bty = "n",
  title = "Outcome"
)
```

Network by Treatment and Outcome



In what follows, we compute *closeness* and *betweenness* centrality measures and visualize it on our network.

Closeness Centrality

Closeness centrality measures how close a node is to all other nodes in the network.

```
# Compute closeness centrality
closeness_centrality <- closeness(
  my_network,
  mode = "all",       # use "in", "out", or "all" for directed graphs
  normalized = TRUE
)

# View results
summary(closeness_centrality)
```

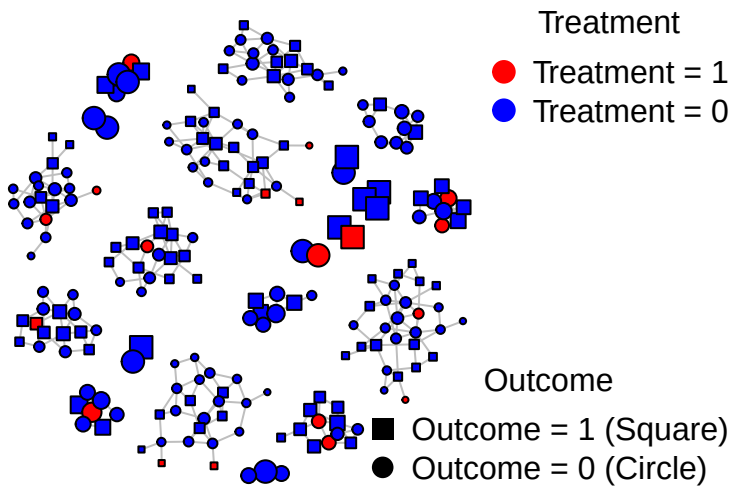
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2727  0.3862  0.4564  0.5069  0.5542  1.0000
```

```
# Add to vertex attributes
V(my_network)$closeness <- closeness_centrality

# Display top 5 nodes by closeness
sort(closeness_centrality, decreasing = TRUE)[1:5]
```

```
## 199 200 202 204 205
##   1   1   1   1   1
```

```
plot(
  my_network,
  vertex.label = NA,
```

```
  vertex.color = V(my_network)$color,
  vertex.shape = V(my_network)$shape,
  vertex.size = 10*closeness_centrality, # proportional to closeness
  edge.width = 1,
  edge.color = "gray",
  layout = layout.coordinates,
  main = "Network by Treatment and Outcome",
  sub = "Node size proportional to closeness"
)

# Legend for treatment (colors)
legend("topright",legend = c("Treatment = 1","Treatment = 0"), col = c("red", "blue"),
       pch = 19, pt.cex = 1.5, bty = "n", title = "Treatment")

# Legend for outcome (shapes)
legend("bottomright",legend = c("Outcome = 1 (Square)","Outcome = 0 (Circle)"),
  pch = c(15, 16),  pt.cex = 1.5, bty = "n", title = "Outcome")
```



Betweenness Centrality

Betweenness centrality measures how often a node lies on shortest paths between other nodes.

```
# Compute betweenness centrality
betweenness_centrality <- betweenness(
  my_network,
  directed = FALSE,    # set TRUE if network is directed
  normalized = TRUE
)
```

```r
# View results
summary(betweenness_centrality)
```

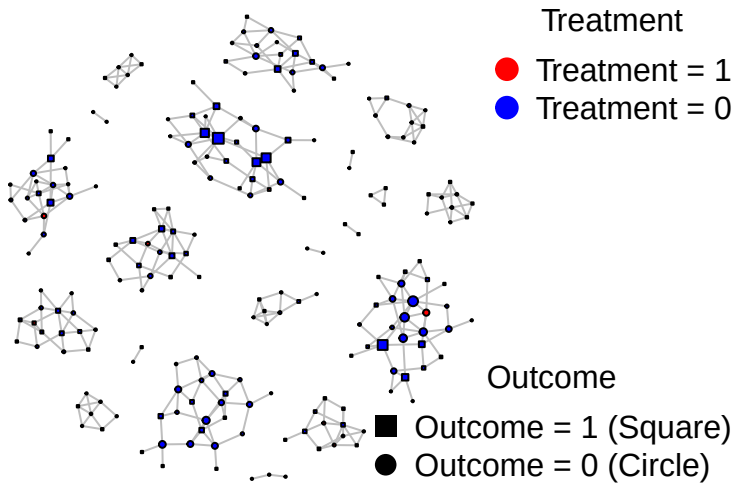
```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.000e+00 1.915e-05 2.094e-04 5.053e-04 6.926e-04 3.678e-03
```

```r
# Add to vertex attributes
V(my_network)$betweenness <- betweenness_centrality

# Display top 5 nodes by betweenness
sort(betweenness_centrality, decreasing = TRUE)[1:5]
```

```
##          90           1           3          99           7
## 0.003677668 0.003283344 0.003258712 0.002901647 0.002736362
```

```r
 plot(
  my_network,
  vertex.label = NA,
  vertex.color = V(my_network)$color,
  vertex.shape = V(my_network)$shape,
  vertex.size = 1+1000*betweenness_centrality, # proportional to betweenness
  edge.width = 1,
  edge.color = "gray",
  layout = layout.coordinates,
  main = "Network by Treatment and Outcome",
  sub = "Node size proportional to betweenness"
)

# Legend for treatment (colors)
legend("topright",legend = c("Treatment = 1","Treatment = 0"), col = c("red", "blue"),
       pch = 19, pt.cex = 1.5, bty = "n", title = "Treatment")

# Legend for outcome (shapes)
legend("bottomright",legend = c("Outcome = 1 (Square)","Outcome = 0 (Circle)"),
  pch = c(15, 16),  pt.cex = 1.5, bty = "n", title = "Outcome")
```

Network by Treatment and Outcome



Node size proportional to betweenness

Hierarchical community detection (edge betweenness)

```
comm_hier <- cluster_edge_betweenness(my_network)
V(my_network)$comm_hier <- membership(comm_hier)
table(V(my_network)$comm_hier )
```

```
##
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
## 28 18 23 19 26 12 15 19 10  7  9  8  6  3  3  2  2  2  2  2
```

Modularity-based community detection (Louvain)

```
comm_louvain <- cluster_louvain(my_network)
V(my_network)$comm_louvain <- membership(comm_louvain)
table(V(my_network)$comm_louvain)
```

```
##
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
## 28 18 23 19 26 12 15 19 10  7  9  8  6  3  3  2  2  2  2  2
```

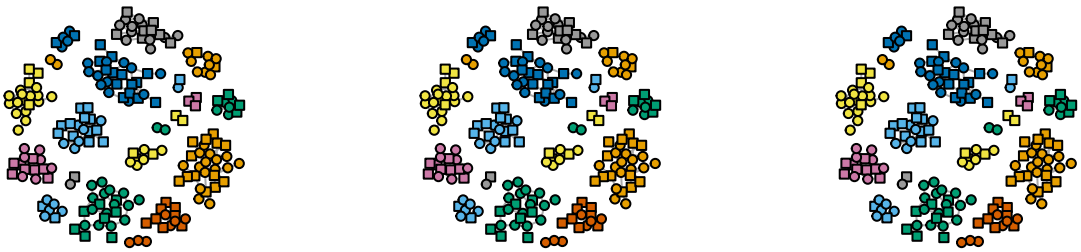
Spectral community detection

```
comm_spectral <- cluster_leading_eigen(my_network)
V(my_network)$comm_spectral <- membership(comm_spectral)
table(V(my_network)$comm_spectral)
```

```
##
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
## 28 18 23 19 26 12 15 19 10  7  9  8  6  3  3  2  2  2  2  2
```

```r
# ------------------------------------------------------------
# VISUALIZATION FUNCTION
# ------------------------------------------------------------
plot_communities <- function(graph, membership, title_text) {

  plot(
    graph,
    vertex.color = membership,   # communities as colors
    vertex.label = NA,
    vertex.size = 8,
    layout = layout.coordinates,
    main = title_text
  )
}
# ------------------------------------------------------------
# Visualizations (3 panels)
# ------------------------------------------------------------

par(mfrow = c(1, 3))  # side-by-side plots

plot_communities(
  my_network,
  V(my_network)$comm_hier,
  "Hierarchical (Edge Betweenness)"
)

plot_communities(
  my_network,
  V(my_network)$comm_louvain,
  "Modularity (Louvain)"
)

plot_communities(
  my_network,
  V(my_network)$comm_spectral,
  "Spectral (Leading Eigenvector)"
)
```

Hierarchical (Edge Betweennes **Modularity (Louvain)** **Spectral (Leading Eigenvecto**



```r
par(mfrow = c(1, 1))  # reset layout

# ------------------------------------------------------------
# Optional: modularity scores comparison
# ------------------------------------------------------------
modularity(comm_hier)
```

```
## [1] 0.9127066
```

```r
modularity(comm_louvain)
```

```
## [1] 0.9127066
```

```r
modularity(comm_spectral)
```

```
## [1] 0.9127066
```

4. Generate random graph

```r
# number of nodes (order)
n<-vcount(my_network)
# number of edges (size)
m<-ecount(my_network)

set.seed(123)
g <- sample_gnm(n = n, m = m, directed = FALSE)

# ------------------------------
# Compute network statistics
# ------------------------------

# Order (number of nodes)
order_g <- vcount(g)

# Size (number of edges)
size_g <- ecount(g)

# Density
density_g <- igraph::edge_density(g)

# Average degree and sd
avg_degree <- mean(igraph::degree(g))
sd_degree <- sd(igraph::degree(g))

# Number of connected components
num_components <- igraph::components(g)$no

# Transitivity (global clustering coefficient)
transitivity_g <- igraph::transitivity(g)

# Degree assortativity
assortativity_g <- igraph::assortativity_degree(g)

# ------------------------------
# Print results
# ------------------------------
```

```r
results <- list(
  Order = order_g,
  Size = size_g,
  Density = density_g,
  Average_Degree = avg_degree,
  SD_Degree = sd_degree,
  Connected_Components = num_components,
  Transitivity = transitivity_g,
  Degree_Assortativity = assortativity_g
)

results
```

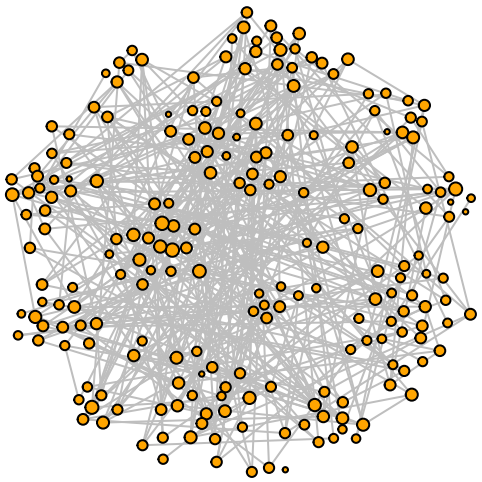
```
## $Order
## [1] 216
##
## $Size
## [1] 352
##
## $Density
## [1] 0.01515935
##
## $Average_Degree
## [1] 3.259259
##
## $SD_Degree
## [1] 1.748729
##
## $Connected_Components
## [1] 8
##
## $Transitivity
## [1] 0.01601423
##
## $Degree_Assortativity
## [1] 0.08587535
```

```r
# Compute closeness centrality
closeness_centrality_g <- closeness(g,mode = "all",normalized = TRUE)
# Compute summary
summary(closeness_centrality_g)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##  0.1466  0.2025  0.2263  0.2243  0.2436  0.2921       7
```

```r
# Note random network contains isolates for which closeness centrality is NA

V(g)$size <- ifelse(
  is.na(closeness_centrality_g),  # check for missing values
  0,                              # replace NA with 0
  20*closeness_centrality_g         # otherwise use centrality value
)

#Visualize random network using my_network coordinates
random_network_plot <- plot(g,
```

```
                        vertex.label = NA,
                        vertex.size = V(g)$size,
                        vertex.color = "orange",
                        edge.width = 1,
                        edge.color = "gray",
                        layout = layout.coordinates)
```

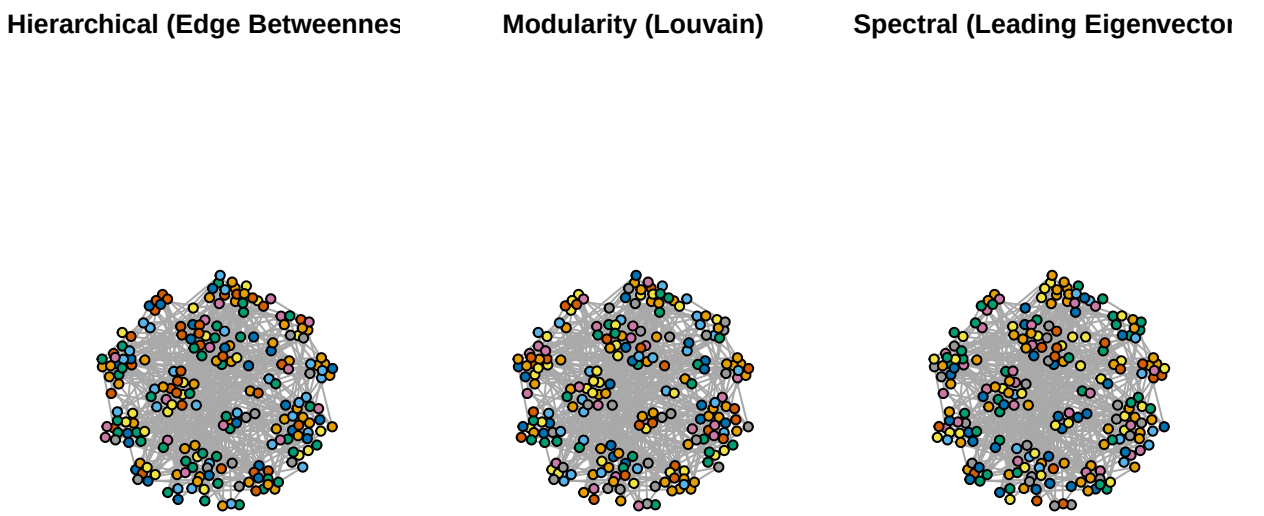


```
random_network_plot
```

```
## NULL
```

```
comm_hier <- cluster_edge_betweenness(g)
V(g)$comm_hier <- membership(comm_hier)

comm_louvain <- cluster_louvain(g)
V(g)$comm_louvain <- membership(comm_louvain)

comm_spectral <- cluster_leading_eigen(g)
V(g)$comm_spectral <- membership(comm_spectral)

# ------------------------------------------------------------
# Visualizations (3 panels)
# ------------------------------------------------------------

par(mfrow = c(1, 3))  # side-by-side plots

plot_communities(
  g,
  V(g)$comm_hier,
  "Hierarchical (Edge Betweenness)"
)

plot_communities(
  g,
  V(g)$comm_louvain,
  "Modularity (Louvain)"
)

plot_communities(
```

```
  g,
  V(g)$comm_spectral,
  "Spectral (Leading Eigenvector)"
)
```




```
par(mfrow = c(1, 1))  # reset layout

modularity(comm_hier)
```

```
## [1] 0.564663
```

```
modularity(comm_louvain)
```

```
## [1] 0.5770919
```

```
modularity(comm_spectral)
```

```
## [1] 0.5242688
```

5. Modeling Network using Exponential Random Graph Models.

Note that you will need to install and load more required packages including "ermg", "networks", "sna", and "coda".

Note that ergm package uses a network of class "network", so we will need to convert our irgaph object my_network to a new network my_network.ergm.

First, extract corresponding adjecency matrix, A. Second, create a new network object of class "network" and build a network object 'my_network.ergm' from adjecency matrix A. Recall, our graph is undirected, so use 'directed=FALSE' option:

```
A <- igraph::as_adjacency_matrix(my_network)
library(ergm)
```

```
## Loading required package: network
```

```
##
## 'network' 1.20.0 (2026-02-06), part of the Statnet Project
## * 'news(package="network")' for changes since last version
## * 'citation("network")' for citation information
## * 'https://statnet.org' for help, support, and other information
```

```
##
```

```
## Attaching package: 'network'

## The following objects are masked from 'package:igraph':
##
##     %c%, %s%, add.edges, add.vertices, delete.edges, delete.vertices,
##     get.edge.attribute, get.edges, get.vertex.attribute, is.bipartite,
##     is.directed, list.edge.attributes, list.vertex.attributes,
##     set.edge.attribute, set.vertex.attribute

##
## 'ergm' 4.12.0 (2026-02-17), part of the Statnet Project
## * 'news(package="ergm")' for changes since last version
## * 'citation("ergm")' for citation information
## * 'https://statnet.org' for help, support, and other information

## 'ergm' 4 is a major update that introduces some backwards-incompatible
## changes. Please type 'news(package="ergm")' for a list of major
## changes.
```

```
library(sna)
```

```
## Loading required package: statnet.common

##
## Attaching package: 'statnet.common'

## The following objects are masked from 'package:base':
##
##     attr, order, replace

## sna: Tools for Social Network Analysis
## Version 2.8 created on 2024-09-07.
## copyright (c) 2005, Carter T. Butts, University of California-Irvine
##  For citation information, type citation("sna").
##  Type help(package="sna") to get started.

##
## Attaching package: 'sna'

## The following objects are masked from 'package:igraph':
##
##     betweenness, bonpow, closeness, components, degree, dyad.census,
##     evcent, hierarchy, is.connected, neighborhood, triad.census
```

```
library(network)
library(coda)

my_network.ergm <- network::as.network(as.matrix(A), directed=FALSE)
my_network.ergm
```

```
##  Network attributes:
##   vertices = 216
##   directed = FALSE
##   hyper = FALSE
##   loops = FALSE
##   multiple = FALSE
##   bipartite = FALSE
##   total edges= 352
##     missing edges= 0
```

```
##     non-missing edges= 352
##
##  Vertex attribute names:
##     vertex.names
##
## No edge attributes
```

Third, setup my_network.ergm attributes

```
nodes$education <- factor(nodes$employment,levels = c(0, 1),labels = c("primary_or_less", "highschool_o
nodes$employment <- factor(nodes$employment,levels = c(0, 1),labels = c("empoyed_searching", "cannotwor
network::set.vertex.attribute(my_network.ergm, "education",as.character(nodes$education))
network::set.vertex.attribute(my_network.ergm, "employment",as.character(nodes$employment))
network::set.vertex.attribute(my_network.ergm, "age",as.numeric(nodes$age))
table(get.vertex.attribute(my_network.ergm, "education"))
```

```
##
## highschool_or_above     primary_or_less
##                  96                 120
```

```
my_network.ergm
```

```
##  Network attributes:
##   vertices = 216
##   directed = FALSE
##   hyper = FALSE
##   loops = FALSE
##   multiple = FALSE
##   bipartite = FALSE
##   total edges= 352
##     missing edges= 0
##     non-missing edges= 352
##
##  Vertex attribute names:
##     age education employment vertex.names
##
## No edge attributes
```

One can also add attributes to edges using the following function network::set.edge.attribute(my_network.ergm,. . .).

Simple Bernoulli ("Erdos/Renyi") model.

Begin with a simple model, containing only the total number of edges in the network: 'edges' - when included in an ERGM its coefficient controls the overall density of the network.

Specify the formula with graph as a dependent variable and term 'edges' as and independent term

```
my.ergm.bern <- formula(my_network.ergm ~ edges)
```

Summary provides the total number of edges in the network

```
summary(my.ergm.bern) # for any such model, you can look at the $g(y)$ statistic for this model
```

```
## edges
##   352
```

Estimate the model using 'ergm' function and look at the fitted model object

```
my.ergm.bern.fit <- ergm(my.ergm.bern)
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.

## Evaluating the predictor and response matrix.

## Maximizing the pseudolikelihood.

## Finished MPLE.

## Evaluating log-likelihood at the estimate.
```

```
summary(my.ergm.bern.fit)
```

```
## Call:
## ergm(formula = my.ergm.bern)
##
## Maximum Likelihood Results:
##
##       Estimate Std. Error MCMC % z value Pr(>|z|)
## edges -4.17386    0.05371      0  -77.71   <1e-04 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 32190  on 23220  degrees of freedom
##  Residual Deviance:  3648  on 23219  degrees of freedom
##
## AIC: 3650  BIC: 3658  (Smaller is better. MC Std. Err. = 0)
```

This simple model specifies a single probability for all ties, captured by the coefficient of the 'edges' term. You can interpret this coefficient in term of the log-odds. The log-odds that a tie is present (two randomly selected companies are conditionally dependent) is

```
coef(my.ergm.bern.fit)
```

```
##     edges
## -4.173863
```

The corresponding probability can be obtained by taking the expit, or inverse logit:

```
exp(coef(my.ergm.bern.fit) )/(1+exp(coef(my.ergm.bern.fit)))
```

```
##      edges
## 0.01515935
```

0.01515935 which is equal to our network density.

Note: you can also compute density using network package

```
gden(my_network.ergm)
```

```
## [1] 0.01515935
```

Models with additional structural terms

Next add a term related to "clustering": the number of completed triangles in the network and look at the summary stats for this model:

```
summary(my_network.ergm ~ edges+triangle)
```

```
##    edges triangle
##      352       84
```

There are 352 edges and 84 closed triangles in this network.

Unfortunately, for our network, this model (my.ergm.triag.fit <- ergm(my_network.ergm ~ edges+triangle))couln't be fitted.For more information on how to interpret coefficient triangle and other coeeficients, see http://statnet.org/Workshops/ergm_tutorial.html#terms_provided_with_ergm

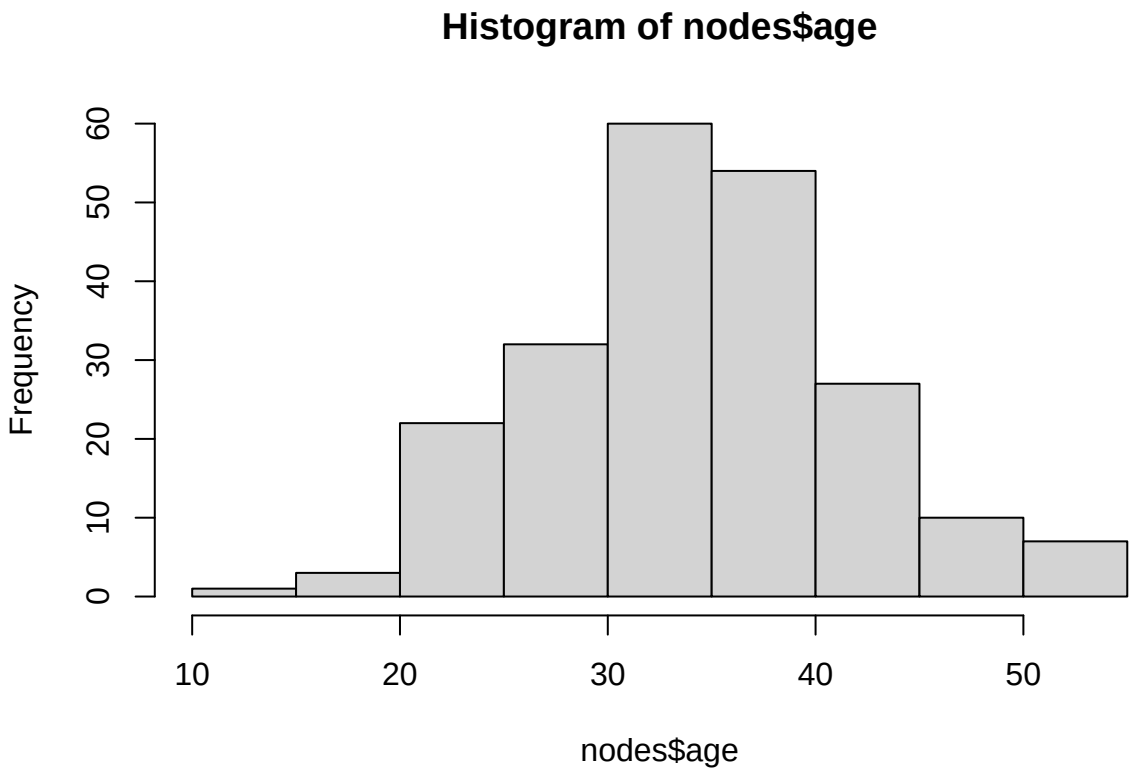
Nodal covariates: effects on mean degree (for numerical covariates)

Age appeared to be associated with higher degree in our network. You can use ergm to test this by including age (numerical) as a nodal covariate via the ergm-term nodecov.

```
summary(nodes$age)  # look at the summary of distribution
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   11.00   30.00   35.00   34.85   40.00   55.00
```

```
hist(nodes$age) # look at the distribution
```




```
summary(my_network.ergm~edges+nodecov('age'))
```

```
##       edges nodecov.age
##         352       24358
```

```
my.ergm.age <- ergm(my_network.ergm~edges+nodecov('age'))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
## Obtaining the responsible dyads.
## Evaluating the predictor and response matrix.
## Maximizing the pseudolikelihood.
## Finished MPLE.
## Evaluating log-likelihood at the estimate.
```

```
summary(my.ergm.age)
```

```
## Call:
## ergm(formula = my_network.ergm ~ edges + nodecov("age"))
##
## Maximum Likelihood Results:
##
##              Estimate Std. Error MCMC % z value Pr(>|z|)
## edges       -3.861990   0.352430      0 -10.958   <1e-04 ***
## nodecov.age -0.004490   0.005033      0  -0.892    0.372
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 32190  on 23220  degrees of freedom
##  Residual Deviance:  3647  on 23218  degrees of freedom
##
## AIC: 3651  BIC: 3667  (Smaller is better. MC Std. Err. = 0)
```

There is a negative, though statistically insignificant, age effect on the probability of a tie formation; that is, lower age is associated with lower log-odds (and hence lower probability) of an edge forming.

```
coef(my.ergm.age)
```

```
##        edges  nodecov.age
## -3.861989599 -0.004490292
```

To interpret the coefficients, note that the Volatility effect operates on both nodes in a edge. The conditional log-odds of a connection between two participants is: -3.861989599 ×change in the number of ties+ (-0.004490292)×the sum of the ages of the two participants, where change in the number of ties=1 for unweighted networks.

For example, the conditional log-odds for a tie between two companies with near minimum age and the corresponding probability of a connection:

```
cond.log.odds<--3.861989599 + (-0.004490292)*2*min(nodes$age)
cond.log.odds
```

```
## [1] -3.960776
```

```
prob.link<-exp(cond.log.odds)/(1+exp(cond.log.odds))
prob.link
```

```
## [1] 0.01869227
```

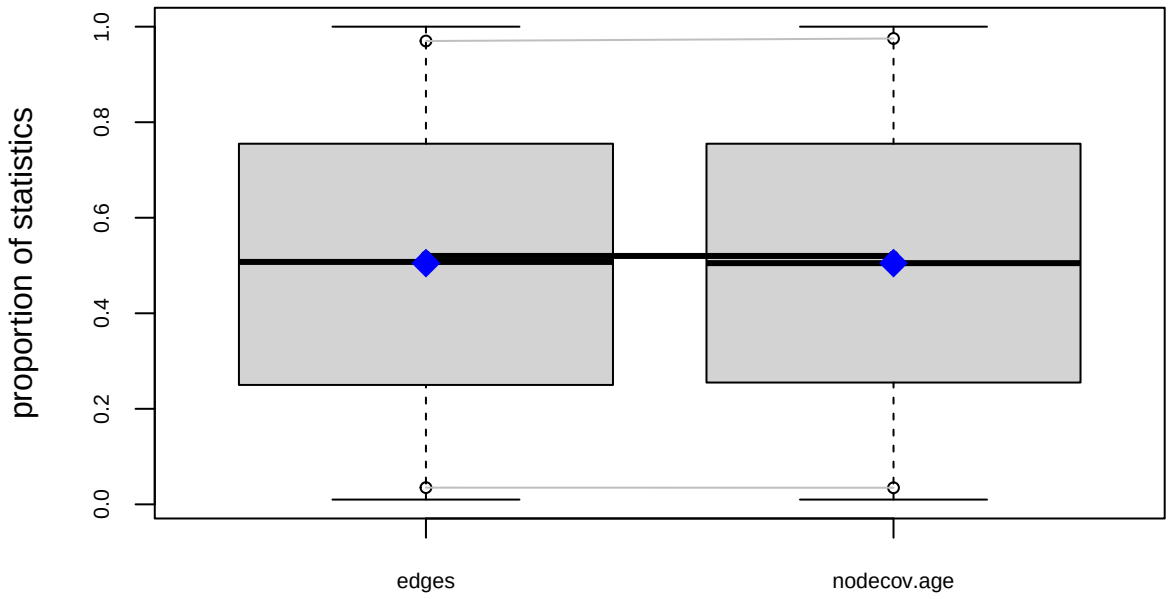
One important step is to check the diagnostic of the model. ERGM diagnostic plots are used to assess how well a fitted exponential random graph model reproduces important structural characteristics of the observed network. The diagnostics compare statistics computed from the observed network to the same statistics computed from networks simulated under the fitted ERGM.

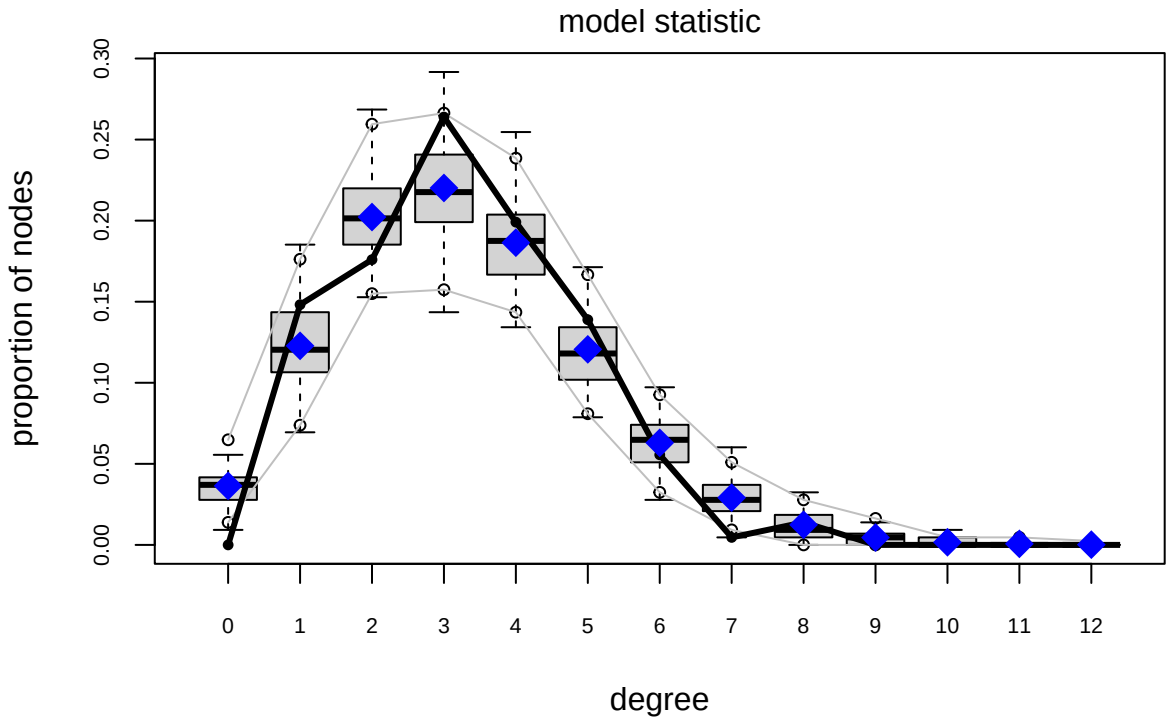
```
my.ergm.gof <- gof(my.ergm.age)
my.ergm.gof
```

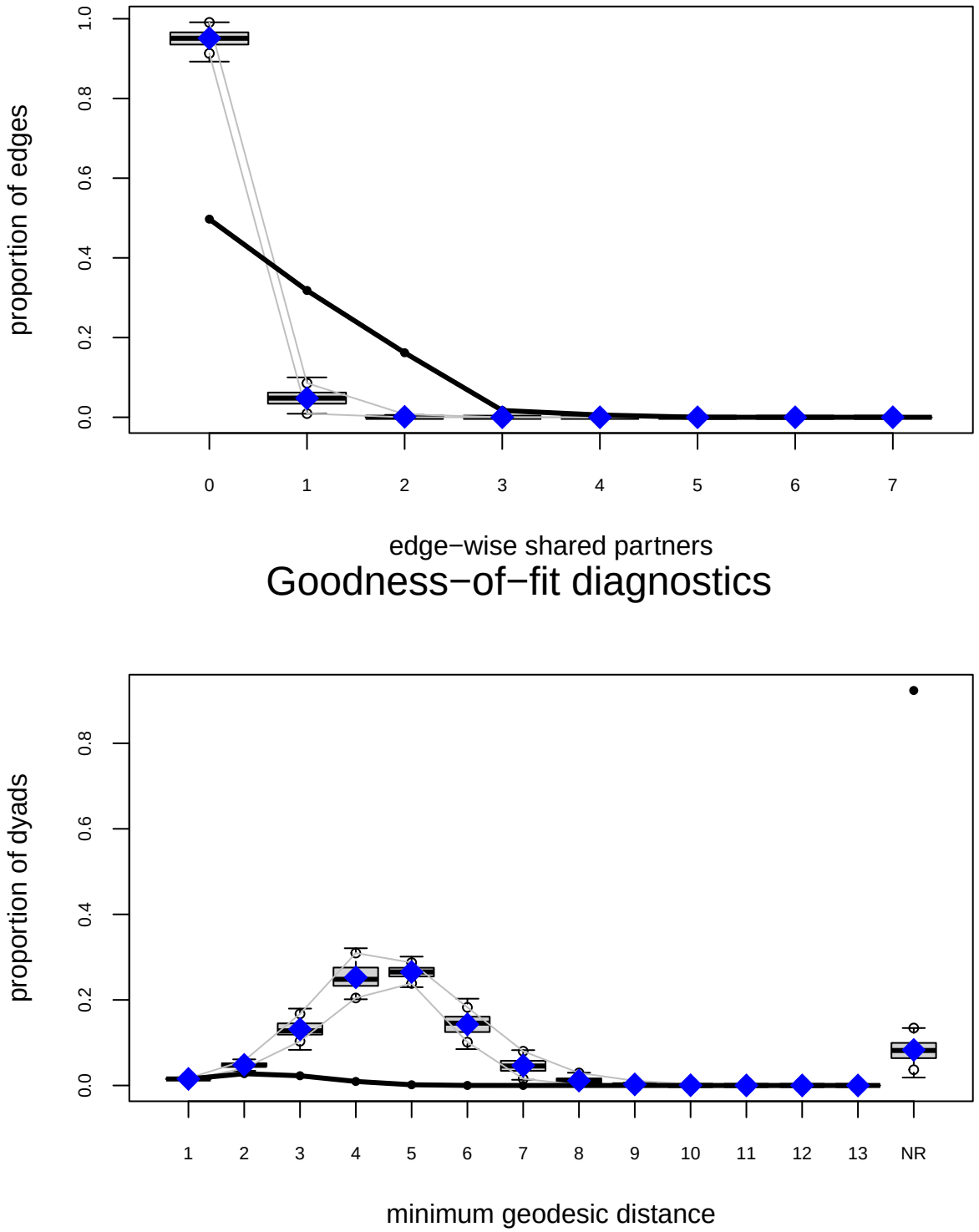
```
##
## Goodness-of-fit for model statistics
##
##               obs   min     mean   max MC p-value
## edges         352   305   354.62   402       1.00
## nodecov.age 24358 21286 24560.67 27874       0.96
##
```

```
## Goodness-of-fit for minimum geodesic distance
##
##       obs  min    mean  max MC p-value
## 1     352  305  354.62  402        1.0
## 2     641  795 1115.50 1417        0.0
## 3     527 1934 3051.94 4174        0.0
## 4     218 3853 5870.92 7449        0.0
## 5      38 5335 6160.98 6995        0.0
## 6       2 1973 3319.05 4710        0.0
## 7       0  308 1079.10 2318        0.0
## 8       0   19  265.08 1025        0.0
## 9       0    0   57.88  547        0.1
## 10      0    0   12.85  321        0.6
## 11      0    0    2.96  130        1.0
## 12      0    0    0.75   43        1.0
## 13      0    0    0.39   25        1.0
## 14      0    0    0.08    6        1.0
## Inf 21442  429 1927.90 4494        0.0
##
## Goodness-of-fit for degree
##
##    obs min  mean max MC p-value
## 0    0   2  7.86  16       0.00
## 1   32  10 26.52  40       0.42
## 2   38  23 43.68  61       0.30
## 3   57  31 47.55  63       0.10
## 4   43  29 40.26  55       0.62
## 5   30  17 26.05  45       0.44
## 6   12   6 13.60  21       0.72
## 7    1   1  6.29  13       0.02
## 8    3   0  2.68   7       0.98
## 9    0   0  0.98   4       0.76
## 10   0   0  0.35   2       1.00
## 11   0   0  0.14   2       1.00
## 12   0   0  0.03   1       1.00
## 13   0   0  0.01   1       1.00
##
## Goodness-of-fit for edgewise shared partner
##
##   obs min   mean max MC p-value
## 0 175 290 337.18 380          0
## 1 112   3  17.01  37          0
## 2  57   0   0.42   5          0
## 3   6   0   0.01   1          0
## 4   2   0   0.00   0          0
```

```
plot(my.ergm.gof)
```


proportion of statistics
0.0
0.2
0.4
0.6
0.8
1.0
edges
nodecov.age


model statistic
proportion of nodes
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0
1
2
3
4
5
6
7
8
9
10
11
12
degree



- In these plots, the boxplots or distributions represent statistics from simulated networks generated by the fitted model, while the observed network statistic is typically shown as a solid line or point. A good model fit is indicated when the observed statistic falls within the range of the simulated distributions.
- Several common network statistics are examined in ERGM diagnostics. The degree distribution evaluates whether the model accurately captures node connectivity patterns. The geodesic distance distribution assesses whether the model reproduces the shortest path structure of the network. Edgewise shared partner (ESP) distributions evaluate clustering and triangle formation, while model statistics assess

how well the ERGM reproduces the structural terms explicitly included in the model specification.

- So geodesic distance and ESP, there is evidence of poor model fit includes observed statistics that consistently fall outside the simulated distributions, substantial discrepancies in the tails of the degree distribution, and failure to reproduce clustering and path-length patterns observed in the empirical network.

There are many more terms to consider:

```
help('ergm-terms')
```